Program 1a

```
from tensorflow.keras.datasets import cifar10
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Use smaller dataset for faster execution
x_train = x_train[:5000]
y_train = y_train[:5000]

# Flatten images
x_train = x_train.reshape(5000, 3072)
x_test = x_test.reshape(10000, 3072)

# Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)

# Train model
knn.fit(x_train, y_train.ravel())

# Predict
y_pred = knn.predict(x_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("KNN Accuracy:", round(accuracy, 2))
```

Prohram 1B

```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense

# Load dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0

# Create 3-layer Neural Network
model = Sequential()

model.add(
   Flatten(
      input_shape=(32, 32, 3)
   )
)

model.add(
   Dense(
      256,
      activation='relu'
   )
)

model.add(
   Dense(
      128,
```

```
        activation='relu'
    )
)

model.add(
    Dense(
        10,
        activation='softmax'
    )
)

# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64
)

# Evaluate model
loss, accuracy = model.evaluate(
    x_test,
    y_test
)
```

```
print(
    "3-Layer Neural Network Accuracy:",
    round(accuracy * 100, 2),
    "%"
)
```

Program 2

```
import numpy as np
from tensorflow.keras.applications.resnet50 import (
    ResNet50,
    preprocess_input,
    decode_predictions
)
from tensorflow.keras.preprocessing import image

# Load pre-trained ResNet50 model
model = ResNet50(weights='imagenet')

# Image path
img_path = "cat.jpg"

# Load image
img = image.load_img(
    img_path,
    target_size=(224, 224)
)

# Convert image to array
img_array = image.img_to_array(img)
```

```
# Add batch dimension
img_array = np.expand_dims(
    img_array,
    axis=0
)

# Preprocess image
img_array = preprocess_input(
    img_array
)

# Predict
predictions = model.predict(
    img_array
)

# Decode prediction
result = decode_predictions(
    predictions,
    top=1
)[0]

print("Prediction:")
print(result)
```

Program 3

```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D
```

```
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize data
x_train = x_train / 255.0
x_test = x_test / 255.0

# Create CNN model
model = Sequential()

model.add(
   Conv2D(
      32,
      (3, 3),
      activation='relu',
      input_shape=(32, 32, 3)
   )
)

model.add(
   MaxPooling2D(
      (2, 2)
   )
)

model.add(
   Flatten()
```

```
)

model.add(
    Dense(
        128,
        activation='relu'
    )
)

model.add(
    Dense(
        10,
        activation='softmax'
    )
)

# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64
)
```

```
# Evaluate model
loss, accuracy = model.evaluate(
    x_test,
    y_test
)

print(
    "CNN Accuracy:",
    round(accuracy * 100, 2),
    "%"
)
```

Program 4

```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize data
x_train = x_train / 255.0
x_test = x_test / 255.0

# Create CNN model
model = Sequential()
```

```
model.add(
    Conv2D(
        32,
        (3, 3),
        activation='relu',
        input_shape=(32, 32, 3)
    )
)

model.add(
    BatchNormalization()
)

model.add(
    MaxPooling2D(
        (2, 2)
    )
)

model.add(
    Dropout(
        0.25
    )
)

model.add(
    Conv2D(
        64,
        (3, 3),
        activation='relu'
```

```
    )
)

model.add(
    MaxPooling2D(
        (2, 2)
    )
)

model.add(
    Flatten()
)

model.add(
    Dense(
        128,
        activation='relu'
    )
)

model.add(
    Dropout(
        0.5
    )
)

model.add(
    Dense(
        10,
        activation='softmax'
    )
```

```
)

# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64
)

# Evaluate model
loss, accuracy = model.evaluate(
    x_test,
    y_test
)

print(
    "Tuned CNN Accuracy:",
    round(accuracy * 100, 2),
    "%"
)
```

Program 5

```
import torch
import torchvision
from PIL import Image
from torchvision import transforms

# Load pre-trained Faster R-CNN model
model = torchvision.models.detection.fasterrcnn_resnet50_fpn(
    pretrained=True
)

model.eval()

# Load image
img = Image.open(
    "object.jpg"
).convert("RGB")

# Transform image to tensor
transform = transforms.ToTensor()

img_tensor = transform(img)

# Predict
with torch.no_grad():
    prediction = model([img_tensor])

print("Objects Detected:")

for i in range(len(prediction[0]["scores"])):
```

```
    score = prediction[0]["scores"][i].item()

    if score > 0.8:

        label = prediction[0]["labels"][i].item()

        print(
            "Object",
            i + 1,
            "Label:",
            label,
            "Confidence:",
            round(score, 2)
        )
```

Program 6

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding
from tensorflow.keras.layers import SimpleRNN
from tensorflow.keras.layers import Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Vocabulary size
max_features = 5000

# Load IMDB dataset
(x_train, y_train), (x_test, y_test) = imdb.load_data(
    num_words=max_features
)
```

```
# Pad sequences
x_train = pad_sequences(
    x_train,
    maxlen=500
)

x_test = pad_sequences(
    x_test,
    maxlen=500
)

# Create RNN model
model = Sequential()

model.add(
    Embedding(
        max_features,
        32
    )
)

model.add(
    SimpleRNN(
        100
    )
)

model.add(
    Dense(
        1,
```

```
        activation='sigmoid'
    )
)

# Compile model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(
    x_train,
    y_train,
    epochs=1,
    batch_size=64
)

# Evaluate model
loss, accuracy = model.evaluate(
    x_test,
    y_test
)

print(
    "Test Accuracy:",
    round(accuracy, 4)
)
```

Program 7

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding
from tensorflow.keras.layers import Bidirectional
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

# Training data
training_data = [
    "hello hi",
    "how are you fine",
    "what is your name chatbot",
    "bye goodbye"
]

# Tokenization
tokenizer = Tokenizer()

tokenizer.fit_on_texts(
    training_data
)

total_words = len(
    tokenizer.word_index
) + 1

# Create sequences
```

```
input_sequences = []

for line in training_data:

    token_list = tokenizer.texts_to_sequences(
        [line]
    )[0]

    input_sequences.append(
        token_list
    )

# Padding
max_len = max(
    len(seq)
    for seq in input_sequences
)

input_sequences = pad_sequences(
    input_sequences,
    maxlen=max_len,
    padding='post'
)

# Split data
X = input_sequences[:, :-1]

y = input_sequences[:, -1]

# One-hot encoding
y = to_categorical(
```

```
    y,
    num_classes=total_words
)

# Build model
model = Sequential()

model.add(
    Embedding(
        total_words,
        64,
        input_length=max_len - 1
    )
)

model.add(
    Bidirectional(
        LSTM(20)
    )
)

model.add(
    Dense(
        total_words,
        activation='softmax'
    )
)

# Compile model
model.compile(
    loss='categorical_crossentropy',
```

```
    optimizer='adam',
    metrics=['accuracy']
)

# Train model
model.fit(
    X,
    y,
    epochs=200,
    verbose=0
)

print("Chatbot Ready!")

# Chat loop
while True:

    user = input("You : ")

    if user.lower() == "exit":

        print("Chatbot : Goodbye")

        break

    elif "hello" in user.lower():

        print("Chatbot : Hi")

    elif "how are you" in user.lower():
```

```
    print("Chatbot : I am fine")

elif "name" in user.lower():

    print("Chatbot : I am a chatbot")

elif "bye" in user.lower():

    print("Chatbot : Goodbye")

else:

    print(
        "Chatbot : Sorry, I don't understand"
    )
```